



PROGRAMAÇÃO AVANÇADA - IA

PROF. JOSENALDE OLIVEIRA

josenalde.oliveira@ufrn.br

<https://github.com/josenalde/ai-advanced-programming>

TÉCNICO EM INFORMÁTICA - EAJ-UFRN

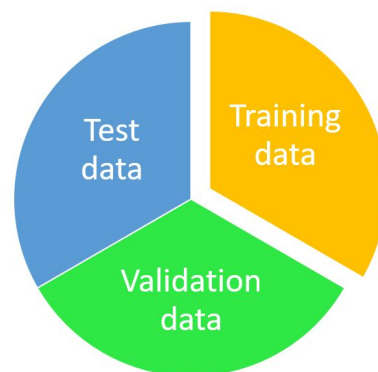
DESAFIOS AO APRENDIZADO DE MÁQUINA

Problemas com o Modelo

[1] Teste e validação

- colocar em produção e monitorar a qualidade pelo feedback?
- o mais simples é dividir em dois conjuntos (80/20, por exemplo), mas depende da quantidade de instâncias no dataset
- a taxa de erro nos casos novos se chama erro de generalização. Ao avaliar o modelo no conjunto de teste se tem uma estimativa deste erro
- se o erro de treinamento for baixo e o de generalização alto, o modelo está sobreajustado aos dados de treinamento.

O problema é que o modelo pode ficar também sobreajustado aos dados de TESTE!

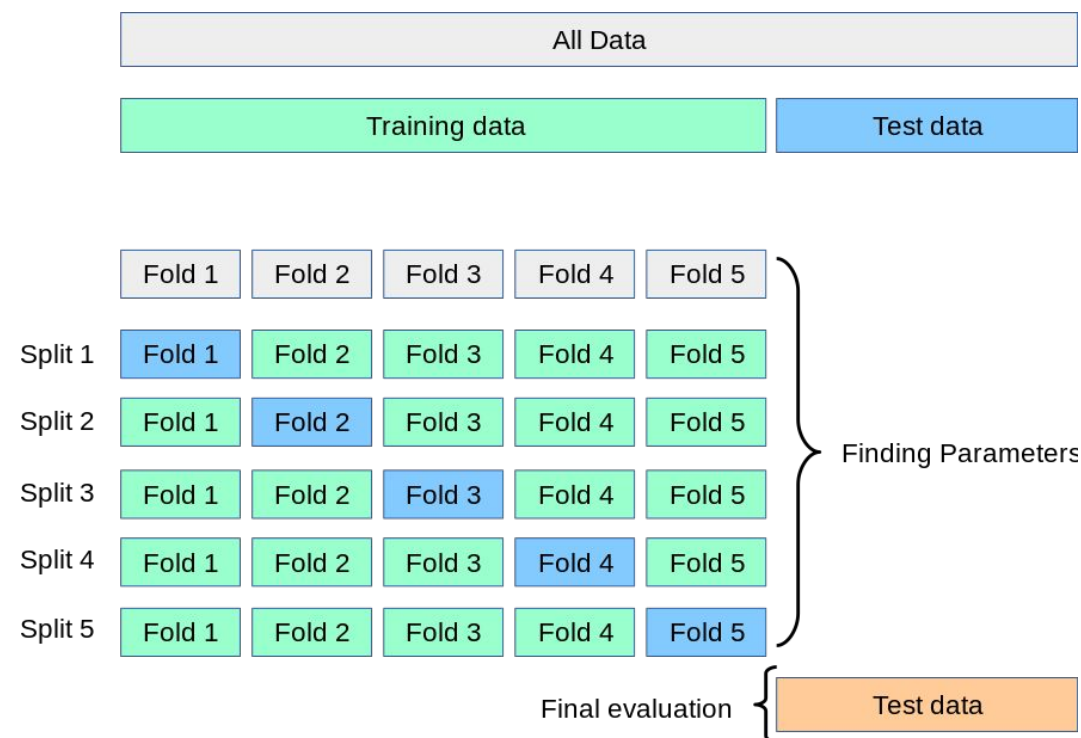
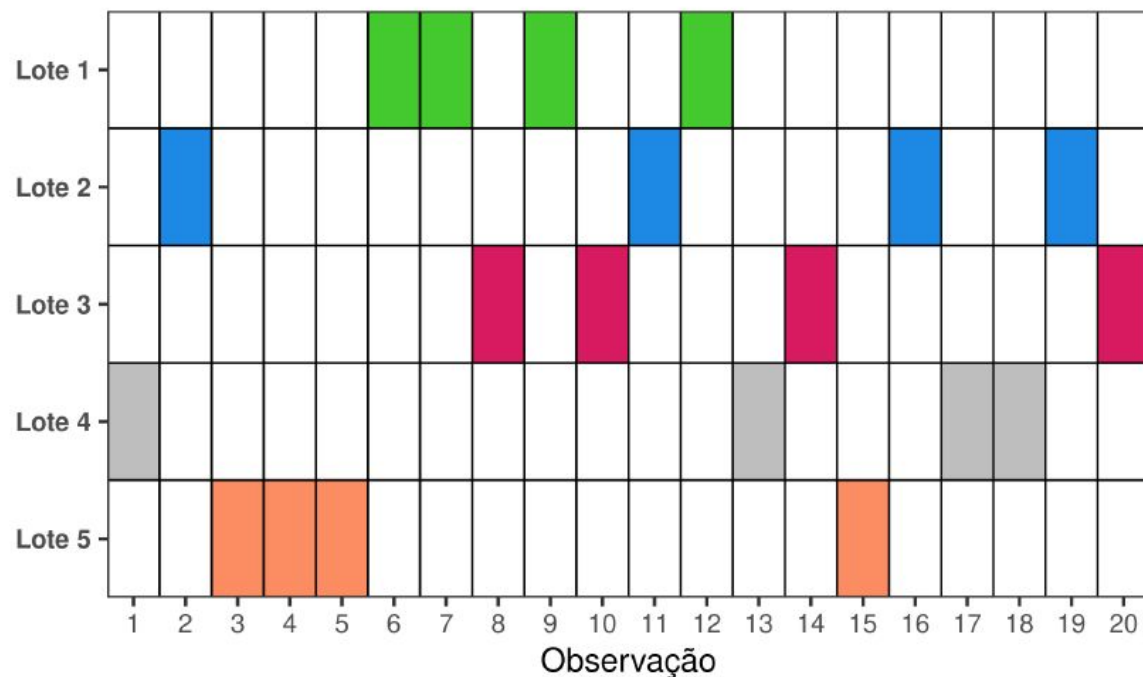


OPÇÕES DE VALIDAÇÃO CRUZADA

K-FOLD

- A ideia é usar vários conjuntos de validação pequenos, onde cada modelo é avaliado uma vez por conjunto de validação após ser treinado no restante dos dados. Tem-se então a média de todas as avaliações, mais preciso, embora mais custoso
- Permite também avaliar a variância da avaliação
- Modelo é treinado em k-1 folds e avaliado no que sobrou. Cada modelo temporário é descartado

5-fold: https://scikit-learn.org/stable/modules/cross_validation.html



K-FOLD: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.cross_val_score.html#sklearn.model_selection.cross_val_score

Exemplo com base didática Titanic

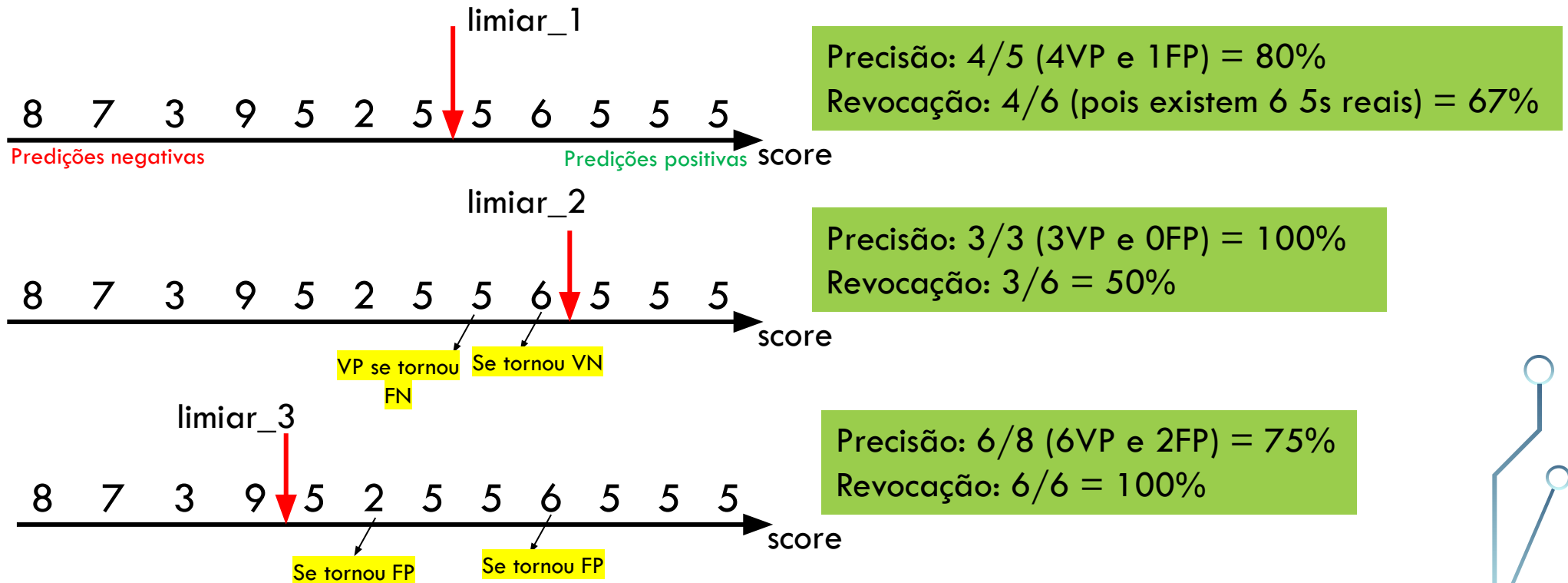
<https://www.kaggle.com/competitions/titanic/data>

A validação cruzada é para avaliar o modelo em dados não vistos e não para gerar um modelo final
Uma vez avaliado a métrica média, treina-se no conjunto completo e aplica-se ao conjunto de teste
Dica de Leitura: [Como treinar o modelo final de machine learning?](#)

A RELAÇÃO PRECISÃO/REVOCAÇÃO

[1] Como um algoritmo de classificação “decide”? Intuitivamente nos parece que é gerada alguma medida interna da relação entre as features (**score**) e, com base em algum limiar (**threshold**) a classificação é atribuída à classe X, Y, Z etc.

[2] O SGDClassifier, por exemplo, para cada instância calcula um **score** baseado em uma **função de decisão** e, se esse score for maior que um limiar, ele atribui a instância à classe positiva, ou então a atribui à classe negativa.



A RELAÇÃO PRECISÃO/REVOCAÇÃO

```
y_scores = cross_val_predict(model_sgd, X_train, y_train_5, cv=5, method='decision_function')

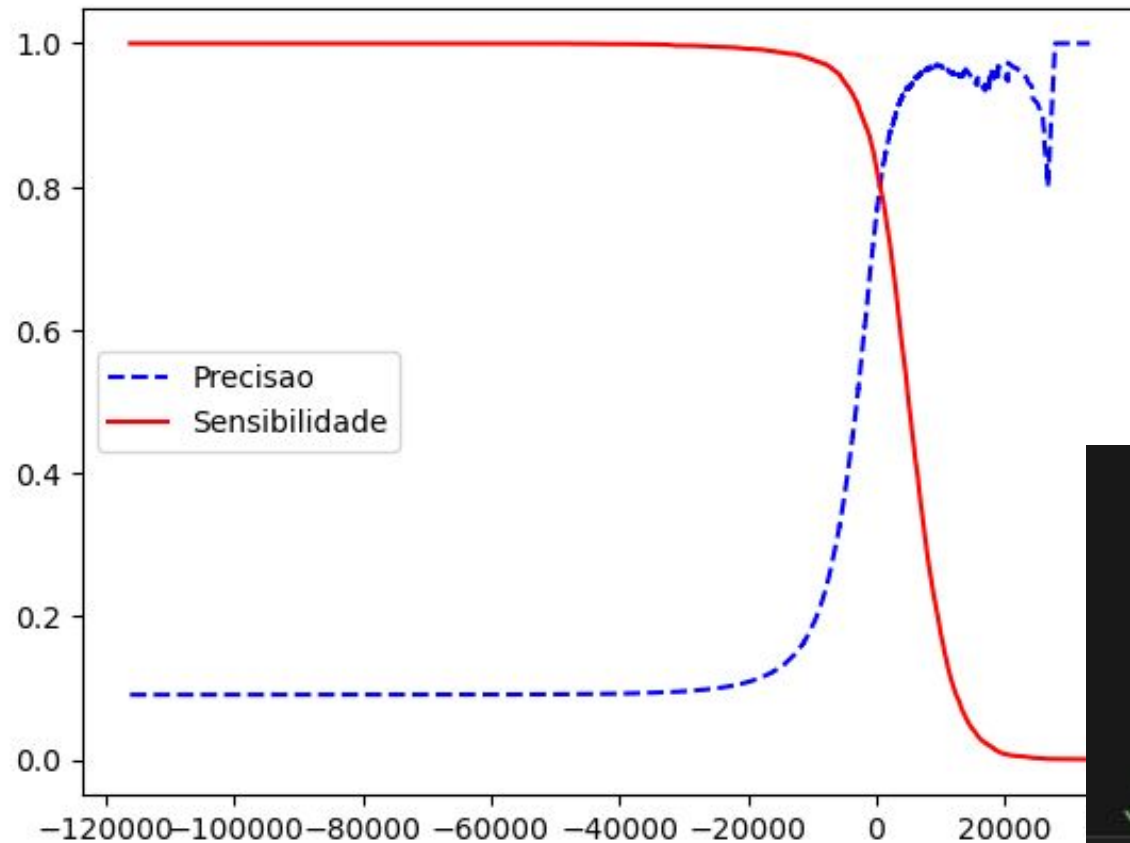
from sklearn.metrics import precision_recall_curve
precisions, recalls, thresholds = precision_recall_curve(y_train_5, y_scores)

def plot_precision_recall_vs_threshold(precisions, recalls, thresholds):
    plt.plot(thresholds, precisions[:-1], 'b--', label='Precisao')
    plt.plot(thresholds, recalls[:-1], 'r-', label='Sensibilidade')
    plt.legend()

plot_precision_recall_vs_threshold(precisions, recalls, thresholds)
plt.show()
```

Classificador de dígito 5

A RELAÇÃO PRECISÃO/REVOCAÇÃO



Para o limiar que dê uma precisão de 90%, qual a revocação associada?

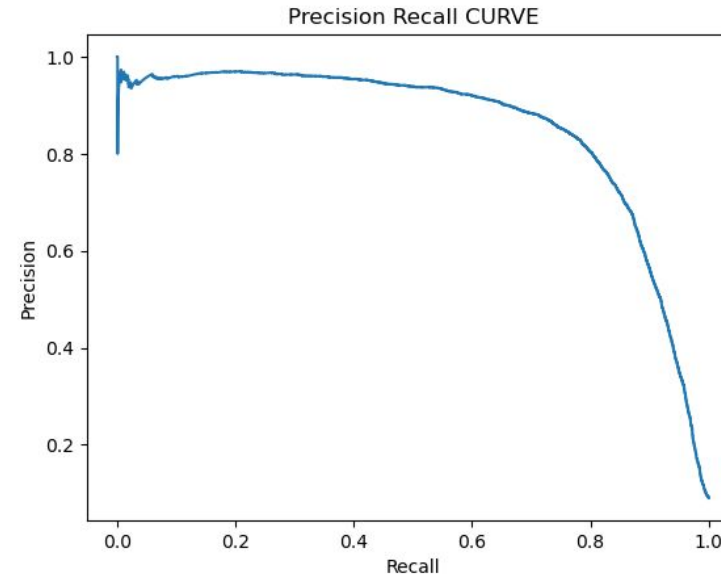
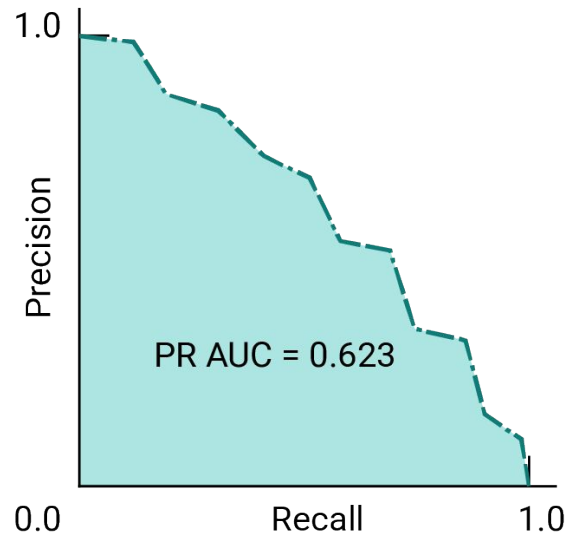
```
threshold_90_precision = thresholds[np.argmax(precisions >= 0.90)]  
print(threshold_90_precision) #3045
```

```
y_train_pred_90 = (y_scores >= threshold_90_precision)  
from sklearn.metrics import precision_score, recall_score  
print(precision_score(y_train_5, y_train_pred_90)) #90%  
print(recall_score(y_train_5, y_train_pred_90)) #65%
```

✓ 0.0s

```
3045.9258227053647  
0.9002016129032258  
0.6589190186312488
```

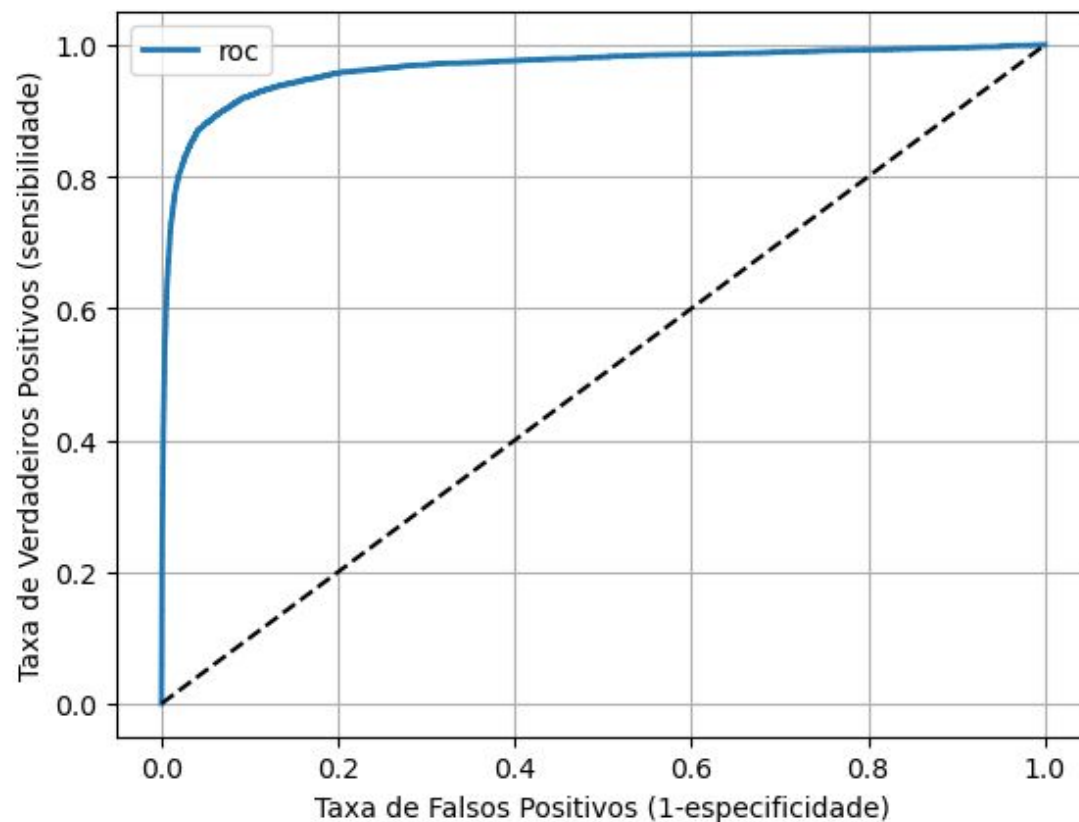
A RELAÇÃO PRECISÃO/REVOCAÇÃO (CURVA PR)



A curva PR e a respectiva área sobre esta curva é mais indicada para bases desbalanceadas

A CURVA ROC (CARACTERÍSTICA DE OPERAÇÃO)

- Muito usada para avaliar classificadores binários
- É a curva entre a Taxa de Verdadeiros Positivos
- (TVP = Revocação = **Sensibilidade**) X Taxa de Falsos Positivos (TFP, taxa de instâncias negativas classificadas como positivas)
- $TFP = 1 - TVN$ ou **$1 - \text{especificidade} = FP/(FP+VN)$**



A área sob a curva (AUC) é uma forma de comparar classificadores (datasets balanceados): (quanto mais próximo de 1, melhor)

A CURVA ROC (CARACTERÍSTICA DE OPERAÇÃO)

```
from sklearn.metrics import roc_curve
tfp, tvp, thresholds = roc_curve(y_train_5, y_scores)

def plot_roc_curve(tfp, tvp, label='roc'):
    plt.plot(tfp, tvp, linewidth=2, label=label)
    plt.plot([0,1], [0,1], 'k--')
    plt.legend()
    plt.grid()
    plt.xlabel('Taxa de Falsos Positivos (1-especificidade)')
    plt.ylabel('Taxa de Verdadeiros Positivos (sensibilidade)')

plot_roc_curve(tfp, tvp)
plt.show()
```

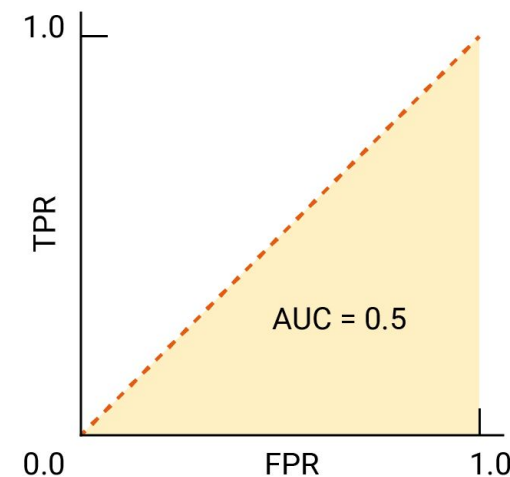
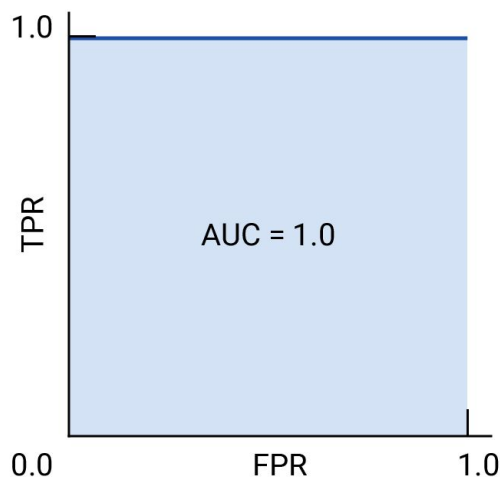
```
from sklearn.metrics import roc_auc_score
roc_auc_score(y_train_5, y_scores)
```

✓ 0.0s

0.9648211175804801

A CURVA ROC (CARACTERÍSTICA DE OPERAÇÃO)

- Em (0,1) temos FPR (TFP=0, ou seja, 1-especificidade=0 implica especificidade=1). Mas lembre que especificidade é a taxa de verdadeiros negativos. Se ela é igual a 1, significa que não há falsos positivos.
- Em (TPR, TVP=1=sensibilidade) não há falsos negativos.
- **Logo é o melhor cenário pontos próximos a (0,1) pois estão os limiares que geram os melhores desempenhos**

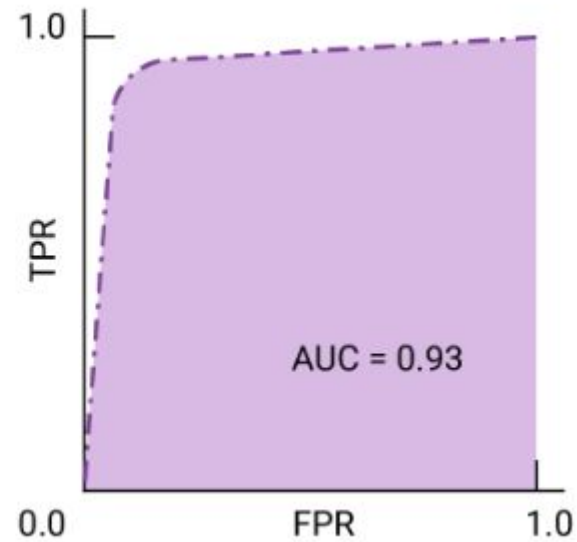
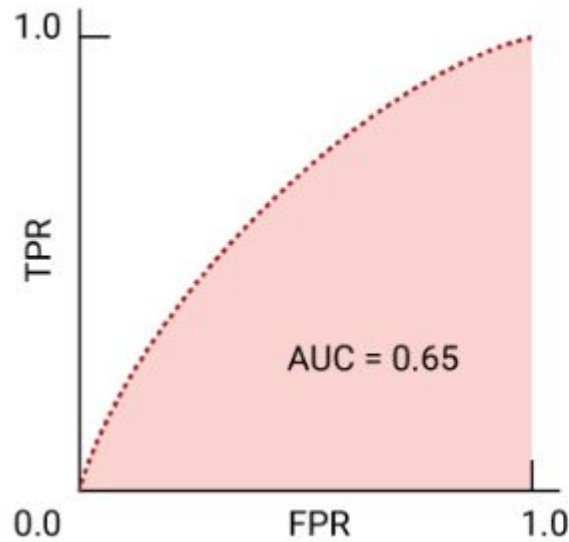


Já neste caso, um AUC de 0.5 equivale a predições aleatórias (como cara-coroa etc.), ou seja, 50% de probabilidade de ser positivo e 50% de ser negativo (1 pra 1)

<https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc>

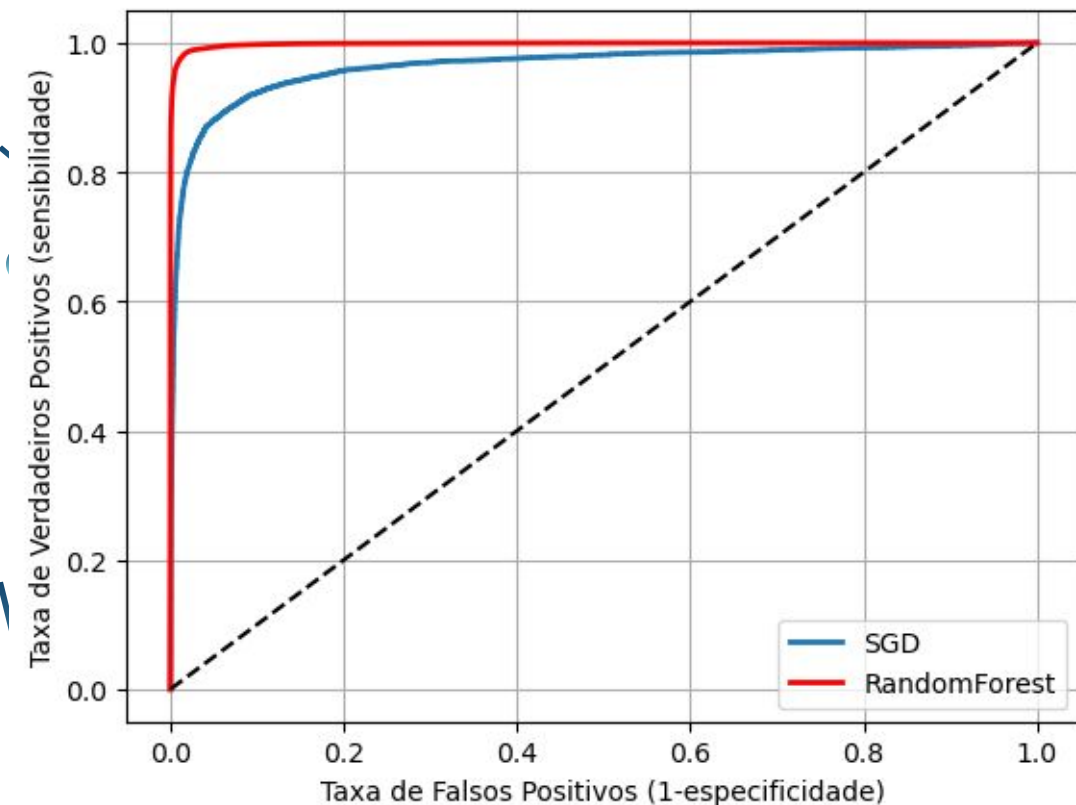
A CURVA ROC (CARACTERÍSTICA DE OPERAÇÃO)

- $0.93 > 0.65$, logo modelo à direita tem melhor desempenho, maior probabilidade de classificar uma amostra aleatória como classe positiva



A CURVA ROC (CARACTERÍSTICA DE OPERAÇÃO)

- Comparando SGD X RandomForest para o classificador binário de 5s no MNIST



```
from sklearn.ensemble import RandomForestClassifier
model_rf = RandomForestClassifier(random_state=42)

y_probab_rf = cross_val_predict(model_rf, X_train, y_train_5, cv=5, method='predict_proba')

y_scores_rf = y_probab_rf[:,1] # ou seja as probabilidades da classe positiva (5)
tfp_rf, tvp_rf, thresholds_rf = roc_curve(y_train_5, y_scores_rf)

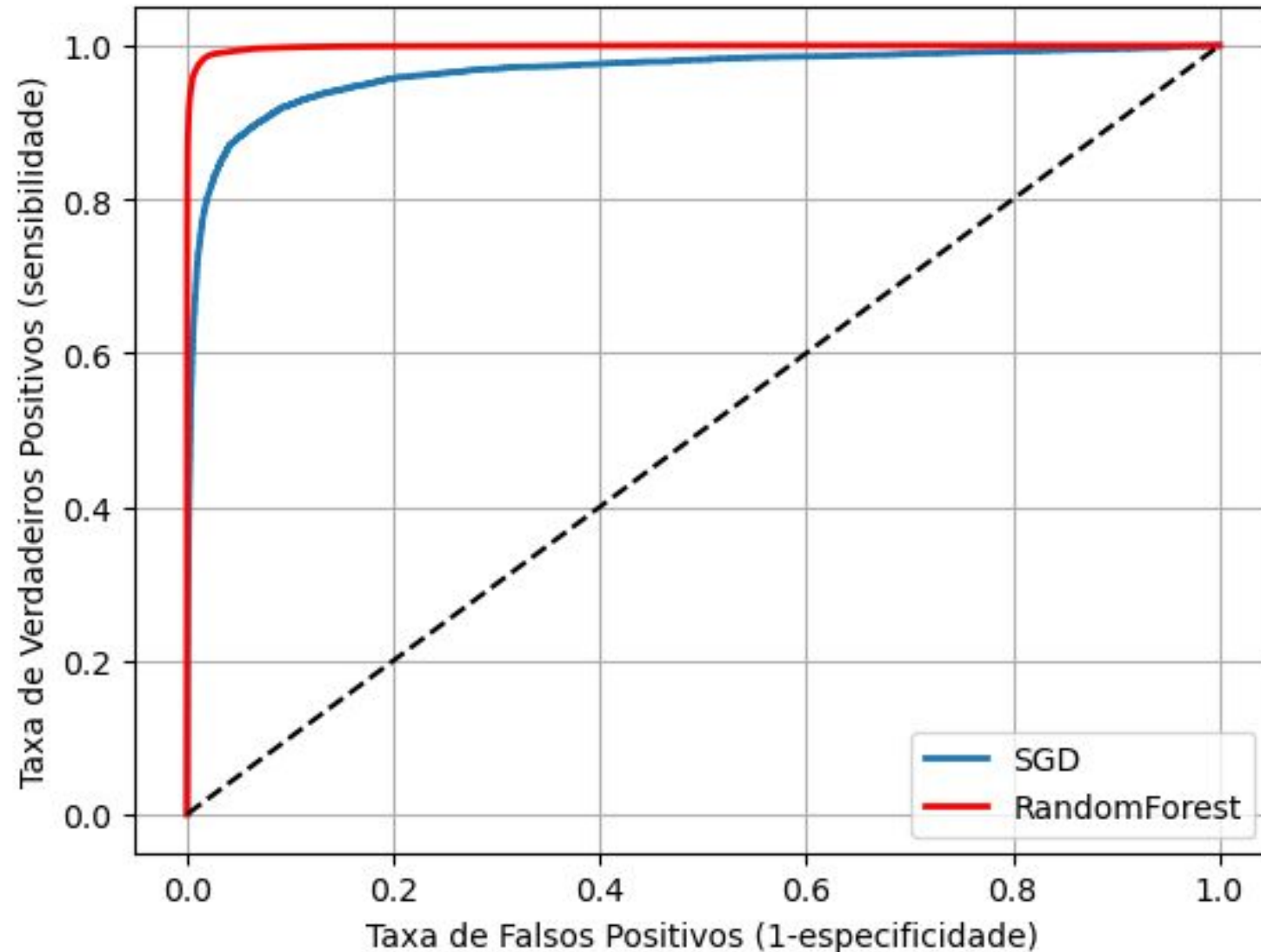
def plot_roc_curve(tfp_1, tvp_1, tfp_2, tvp_2, label_1, label_2):
    plt.plot(tfp_1, tvp_1, linewidth=2, label=label_1)
    plt.plot(tfp_2, tvp_2, 'r', linewidth=2, label=label_2)
    plt.plot([0,1],[0,1], 'k--')
    plt.legend()
    plt.grid()
    plt.xlabel('Taxa de Falsos Positivos (1-especificidade)')
    plt.ylabel('Taxa de Verdadeiros Positivos (sensibilidade)')

plot_roc_curve(tfp, tvp, tfp_rf, tvp_rf, 'SGD', 'RandomForest')
plt.show()
```

ROC AUC_RF = 0.9984

A CURVA ROC (CARACTERÍSTICA DE OPERAÇÃO)

- Comparando SGD X RandomForest para o classificador binário de 5s no MNIST



ROC AUC_RF = 0.9984

APERFEIÇOANDO O MODELO

- A escolha manual de hiperparâmetros é tediosa (encontrar uma combinação)
- No Scikit-learn pode-se usar o GridSearchCV ou o RandomizedSearchCV
- No GridSearch ele avalia TODAS as combinações por meio de validação cruzada
- Exemplo para o RandomForest

```
from sklearn.model_selection import GridSearchCV
hyperparam_grid = [{'n_estimators':[3,10,30],
                    'max_features':[2,4,6,8]},
                   {'bootstrap':[False], 'n_estimators':[3,10], 'max_features':[2,3,4]},]
model_rf = RandomForestRegressor()
model_rf_gs = GridSearchCV(model_rf, hyperparam_grid,
                           cv=5,scoring='f1',return_train_score=True)
model_rf_gs.fit(X,y)
```

APERFEIÇOANDO O MODELO

Testa no primeiro dict $3 \times 4 = 12$ combinações com o parâmetro `bootstrap=True` (default)

Depois testa no segundo dict $2 \times 3 = 6$ combinações com `bootstrap=False`

Testa portanto $12 + 6$ combinações e cada rodada de 5 treinos com validação cruzada, ou seja,

$18 \times 5 = 90$ rodadas de treinamento. Ao fim, os melhores parâmetros estão aqui:

```
model_rf_gs.best_params_  
# se os melhores resultados forem com os limites superiores, pense em aumenta-los  
model_rf_gs.best_estimator_ (melhor estimador)  
cvres = model_rf_gs.cv_results_  
for mean_f1, params in zip(cvres['f1'], cvres['params']):  
    print(mean_f1, params)
```

APERFEIÇOANDO O MODELO

- Outro exemplo com KNN

```
from sklearn.model_selection import GridSearchCV
```

```
param_grid = {'n_neighbors': range(1,40,2), 'weights': ['uniform', 'distance'], 'p': [1, 2, 3]}  
grid = GridSearchCV(KNeighborsClassifier(),param_grid, verbose = 3)#verbose indica a quantidade de detalhame  
grid.fit(X_train,y_train)
```

Então você pode executar previsões neste objeto da grade com o conjunto de teste

```
grid_predictions = grid.predict(X_test)
```

```
dfGridSearch = pd.DataFrame(grid.cv_results_)
```

```
dfGridSearch.loc[dfGridSearch['rank_test_score'] == 1, :]
```

APERFEIÇOANDO O MODELO

- Se o espaço de pesquisa for grande é melhor usar busca randomizada
- Avalia determinado número de combinações aleatórias, selecionando um valor aleatório para cada hiperparâmetro por iteração (1000 iterações no Randomized explorar mais do que no GridSearch) e pode-se definir o número de iterações desejado para controlar melhor o custo computacional

```
from sklearn.model_selection import RandomizedSearchCV
hyperparam_grid = [{ 'n_estimators':[3,10,30],
                     'max_features':[2,4,6,8]},
                   { 'bootstrap':[False], 'n_estimators':[3,10], 'max_features':[2,3,4]},]
model_rf = RandomForestRegressor()
model_rf_gs = RandomizedSearchCV(model_rf, hyperparam_grid, n_iter=10, scoring='f1', cv=5,
return_train_score=True)
model_rf_gs.fit(X,y)
```